Programarea calculatoarelor

FMI

Secția Calculatoare și tehnologia informației, anul I

Cursul 5 / 03.11.2025

Programa cursului

1. Introducere
- Algoritmi
- Limbaje de programare.
- Introducere în limbajul C. Structura unui program C

2. Fundamentele limbajului C
- Etapele realizării unui program C
- Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii
- Tipuri derivate de date: tablouri, șiruri de caractere, structuri, uniuni, câmpuri de biți, enumerări, pointeri
- Instrucțiuni de control
- **Directive de preprocesare. Macrodefiniții**
- **Funcții de citire/scriere**

3. Fișiere text
- Funcții specifice de manipulare

4. Funcții (1)
- Declarare și definire. Apel.
- Metode de trasmitere a paramerilor.
- Pointeri la funcții

5. Tablouri și pointeri
- Legătura dintre tablouri și pointeri
- Aritmetica pointerilor
- Alocarea dinamică a memoriei
- Clase de memorare

6. Șiruri de caractere
- Funcții specifice de manipulare

7. Fișiere binare
- Funcții specifice de manipulare

8. Structuri de date complexe și autoreferite
- Definire și utilizare

9. Funcții (2)
- Funcții cu număr variabil de argumente
- Preluarea argumentelor funcției main din linia de comandă
- Programare generică

Cuprinsul cursului de azi

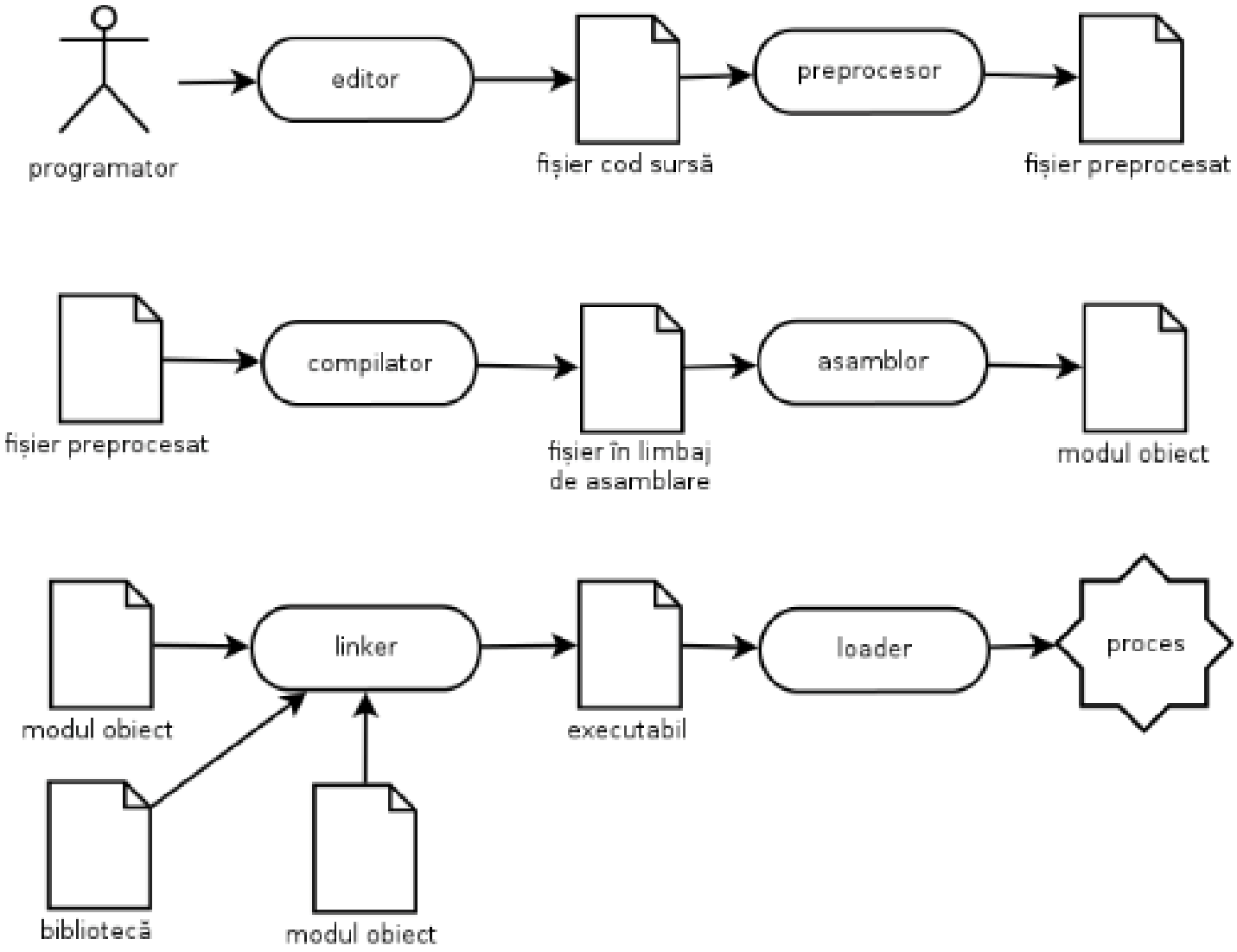
1. Directive de preprocesare. Macrodefiniții
2. Funcții de citire/ scriere

Directive de preprocesare. Macrodefiniții

Cuprins:

- Etapele realizării unui program. Preprocesarea
- Directiva #include
- Directiva #define
- Define vs typedef. Macro vs funcție
- Compilarea condiționată: #if, # ifdef, #ifndef
- Macrouri predefinite

Etapele realizării unui program în C



Etapele realizării unui program în C

Etape pentru obținerea unui cod executabil:

1. **editarea** codului sursă
   - ✓ salvarea fișierului cu extensia *.c*
2. **preprocesarea**
   - ✓ efectuarea directivelor de preprocesare (**#include**, **#define**)
   - ✓ ca un editor – modifică și adaugă la codul sursă
3. **compilarea**
   - ✓ verificarea sintaxei
   - ✓ codul este tradus din cod de nivel înalt în limbaj de asamblare
4. **asamblarea**
   - ✓ transformare în cod obiect (limbaj mașină) cu extensia .o, .obj
     - ✓ nu este încă executabil !
5. **link-editarea** (editarea legăturilor)
   - ✓ combinarea codului obiect cu alte coduri obiect (al bibliotecilor asociate fișierelor header)
   - ✓ transformarea adreselor simbolice în adrese reale

Preprocesarea în limbajul C

- apare înaintea procesului de compilare a codului sursă (fișier text editat într-un editor și salvat cu extensia .c)
- preprocesarea codului sursă asigură:
  - ✓ **includerea conținutului fișierelor (de obicei a fișierelor *header*)**
  - ✓ **definirea de macrouri (macrodefiniții)**
  - ✓ **compilarea condiționată**
- constă în substituirea simbolurilor din codul sursă pe baza directivelor de preprocesare
- directivele de preprocesare sunt precedate de caracterul diez **#**

Directiva #include

- copiază conținutul fișierului specificat în textul sursă

- **#include <nume_fisier>**
  - ✓ caută *nume_fisier* în directorul unde se află fișierele din librăria standard instalată odată cu compilatorul

- **#include “nume_fisier”**
  - ✓ caută *nume_fisier* în directorul curent

Directiva #include. *Exemple*

```
// include fisierul stdio.h
// din directoarele standard
#include <stdio.h>

// include fiserul Liste.h;
// cautarea se face intai in directorul
// curent si dupa aceea in directoarele standard
#include “Liste.h"


// include fisierul Masive.cpp din directorul
// c:\Biblioteci; daca fisierul nu exista nu
// mai este cautat in alta parte si se genereaza
// o eroare de compilare
#include "C:\Biblioteci\Masive.cpp"
```

Directiva #define

- folosită pentru definirea (înlocuirea) constantelor simbolice și a macrourilor
- definirea unei **constante simbolice** este un caz special al definirii unui macro

```
#define nume   text
```

  - ✓ în timpul preprocesării **nume** este înlocuit cu **text**
- **text** poate să fie mai lung decât o linie, continuarea se poate face prin caracterul \ pus la sfârșitul liniei
- **text** poate să lipsească, caz în care se definește o constantă vidă

Directivele **#define** și **#include.** *Exemplu*

Ce afișează programul?

main.c | ceva.txt

```
1  int divizor(int a, int b)
2  {
3      int c=a%b;
4      while(c)
5      { a=b; b=c; c=a%b;}
6      return b;
7  }
```

main.c | ceva.txt

```
1  #include <stdio.h>
2  #include "ceva.txt"
3  #define MIN 0
4
5  int main()
6  {
7      int a,b;
8      do
9      {
10         printf("a="); scanf("%d", &a);
11         printf("b="); scanf("%d", &b);
12     }while (a<=MIN || b<=MIN);
13     printf("cmmdc(%d,%d)= %d\n",a,b,divizor(a,b));
14     return 0;
15 }
```

Directiva **#define.** *Exemplu*

```c
#include <stdio.h>
#define DIM_VECTOR 20   // definire parametru
#define TIP double      // definire tip vector
#define MESAJ "Suma:\n" // definire mesaj
// definire cod pe mai multe linii:
#define SEPARATOR printf( \
        "---------------" \
       "\n");

int main()
{
    /*declarare vector folosind simbolurile specificate*/
    TIP v[DIM_VECTOR] = {1.1, 3.23, 6.62};
    int n = 3;

    printf(MESAJ);  //utilizare simboluri pentru mesaj
    SEPARATOR;  //inserare cod prin preprocesor
    //validare lungime:
    if (n > DIM_VECTOR)
        printf("dimensiunea este mai mare ca DIM_VECTOR\n");
    //calcul suma:
    TIP s = 0;
    for (int i = 0; i < n; i++)
       s += v[i];
    printf("%.4f\n", s);

return 0;
}
```

Directiva **#define.** *Exemplu*

```c
#include <stdio.h>
#define DIM_VECTOR 20   // definire parametru
#define TIP double      // definire tip vector
#define MESAJ "Suma:\n" // definire mesaj
// definire cod pe mai multe linii:
#define SEPARATOR printf( \
        "---------------" \
      "\n");

int main()
{
    /*declarare vector folosind simbolurile specificate*/
    TIP v[DIM_VECTOR] = {1.1, 3.23, 6.62};
    int n = 3;

    printf(MESAJ);  //utilizare simboluri pentru mesaj
    SEPARATOR;  //inserare cod prin preprocesor
    //validare lungime:
    if (n > DIM_VECTOR)
        printf("dimensiunea este mai mare ca DIM_VECTOR\n");
    //calcul suma:
    TIP s = 0;
    for (int i = 0; i < n; i++)
       s += v[i];
    printf("%.4f\n", s);

return 0;
}
```

```
Suma:
---------------
10.9500
```

Directiva **#define.** *Exemplu*

```c
#include <stdio.h>
#define DIM_VECTOR 20   // definire parametru
#define TIP double      // definire tip vector
#define MESAJ "\tSuma:\n" // definire mesaj
// definire cod pe mai multe linii:
#define SEPARATOR printf( \
        "\t---------------" \
       "\n");

TIP suma(TIP v[], int n)
{
    //validare lungime:
    if (n > DIM_VECTOR)
        printf("dimensiunea este mai mare ca DIM_VECTOR\n");
    //calcul suma:
    TIP suma = 0;
    for (int i = 0; i < n; i++)
        suma += v[i];
    return suma;
}

int main()
{
    /*declarare vector folosind simbolurile specificate*/
    TIP v[DIM_VECTOR] = {1.1, 3.23, 6.62};
    int n = 3;

    printf(MESAJ);  //utilizare simboluri pentru mesaj
    SEPARATOR;  //inserare cod prin preprocesor
    //apel functie
    printf("\t%.4f",suma(v, 3));

return 0;
}
```

```
Suma:
---------------
10.9500
```

Directiva #define

`#define nume text`

- înlocuirea se continuă până în momentul în care **nume** nu mai este definit sau până la sfârșitul fișierului

- renunțarea la definirea unei constante simbolice se poate face cu directiva **`#undef nume`**

Typedef

- asociază un nume unui tip de date, predefinit sau definit de programator

Sintaxa:

typedef descriere_tip nume_tip;

Exemple:

```
typedef float real;
typedef int* PInt;
typedef int vector[20];
typedef struct {
                int grad;
                double coeficient[100];} polinom;
```

#define vs typedef

#define	typedef
- Poate să definească aliasuri și pentru valori, de exemplu: 1 ca ONE, 3.14 ca PI etc	- dă nume simbolice numai tipurilor. **Definește un nou tip prin copierea și lipirea definițiilor câmpurilor unui struct**
- instrucțiunile #define sunt interpretate de preprocesor	- instrucțiunile typedef sunt interpretate de compilator
- Nu trebuie să existe punct și virgulă la sfârșitul lui #define	- Se scrie punct și virgulă la sfârșitul lui typedef

Directiva #define

- definirea unui **macro**:

```
#define nume(lista-parametri) text
```

  - ✓ numele macro-ului este **nume**
  - ✓ lista de parametri este de ex.: **p1, p2, …, pn**
  - ✓ textul substituit este **text**

- parametrii formali sunt substituiți de cei actuali în text
- apelul macro-ului este similar apelului unei funcții:

```
nume(p_actual1,p_actual2,...,p_actualn)
```

Directiva **#define.** *Exemple*

```
1  #include <stdio.h>
2  #define SQUARE(x) x*x
3
4  int main()
5  {
6      int a=3;
7      printf("%d, %d\n",SQUARE(a),a);
8      printf("%d, %d\n",SQUARE(a++),a);
9      printf("%d, %d\n",SQUARE(a+1),a);
10
11     return 0;
12 }
```

Ce afișează programul?

Directiva **#define.** *Exemple*

```c
#include <stdio.h>
#define SQUARE(x) x*x

int main()
{
    int a=3;
    printf("%d, %d\n",SQUARE(a),a);
    printf("%d, %d\n",SQUARE(a++),a);
    printf("%d, %d\n",SQUARE(a+1),a);

    return 0;
}
```

```
9, 3
12, 5
11, 5
```

Explicația în următorul diapozitiv.

Directiva #define

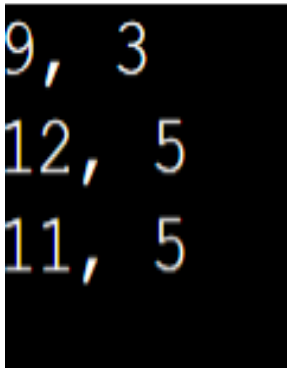
#define SQUARE(x) x * x

- Dacă x=3 și apelăm **SQUARE(x)**, atunci în program preprocesorul substituie: **3 * 3**
- Dacă x=3 și apelăm **SQUARE(x++)**, atunci:

  x++ * (x++) = 3 * 4 = 12, iar x = 5

- Dacă x=5 și apelăm **SQUARE(x+1)**, atunci:

  x + 1 * x + 1 = 5 + 1 * 5 + 1 = 11



Directiva #define

Corecție:

#define SQUARE(x) (x) * (x)

Când apelăm:

printf(“%d \n”, SQUARE(a + 1))

preprocesorul înlocuiește:

printf(“%d \n”, (a + 1)*(a + 1))

Directiva **#define.** *Exemple*

```c
#include <stdio.h>
#define SQUARE(x) x*x
#define Square(x) (x)*(x)
int main()
{
    int a=3;
    printf("%d, %d\n",SQUARE(a),a);
    printf("%d, %d\n",SQUARE(a++),a);
    printf("%d, %d\n",SQUARE(a+1),a);
    int b=3;
    printf("%d, %d\n",Square(b),b);
    printf("%d, %d\n",Square(b++),b);
    printf("%d, %d\n",Square(b+1),b);

    return 0;
}
```

Ce afișează programul?

Directiva **#define.** *Exemple*

```c
#include <stdio.h>
#define SQUARE(x) x*x
#define Square(x) (x)*(x)
int main()
{
    int a=3;
    printf("%d, %d\n",SQUARE(a),a);
    printf("%d, %d\n",SQUARE(a++),a);//a++*(a++)
    printf("%d, %d\n",SQUARE(a+1),a);//a+1*a+1
    int b=3;
    printf("%d, %d\n",Square(b),b);
    printf("%d, %d\n",Square(b++),b);//b++*(b++)
    printf("%d, %d\n",Square(b+1),b);//(b+1)*(b+1)

    return 0;
}
```

```
9, 3
12, 5
11, 5
9, 3
12, 5
36, 5
```

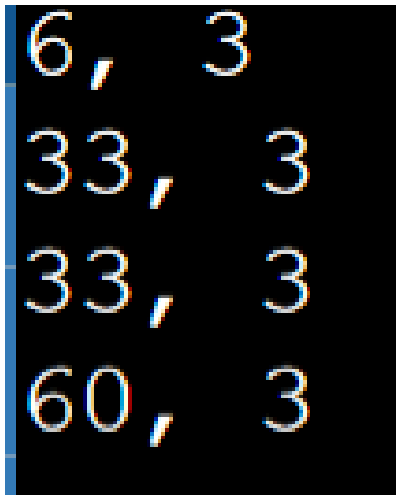
Directiva #define

```
#include <stdio.h>
#define DOUBLE(x) x+x
#define Double(x) (x)+(x)
#define DoublE(x) (x+x)
int main()
{
    int a=3;
    printf("%d, %d\n", DOUBLE(a),a);
    printf("%d, %d\n",10*DOUBLE(a),a);
    printf("%d, %d\n",10*Double(a),a);
    printf("%d, %d\n",10*DoublE(a),a);
    return 0;}
```

Ce afișează programul?

Directiva #define

```
#include <stdio.h>
#define DOUBLE(x) x+x
#define Double(x) (x)+(x)
#define DoublE(x) (x+x)
int main()
{
    int a=3;
    printf("%d, %d\n", DOUBLE(a),a); //a+a
    printf("%d, %d\n",10*DOUBLE(a),a); //10*a+a
    printf("%d, %d\n",10*Double(a),a); //10*a+a
    printf("%d, %d\n",10*DoublE(a),a); //10*(a+a)
    return 0;}
```



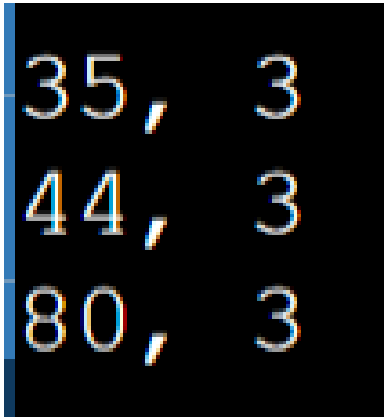
Directiva #define

Ce afișează programul?

```
#include <stdio.h>
#define DOUBLE(x) x+x
#define Double(x) (x)+(x)
#define DoublE(x) (x+x)
int main()
{
   int a=3;
   printf("%d, %d\n",10*DOUBLE(a+1),a);
   printf("%d, %d\n",10*Double(a+1),a);
   printf("%d, %d\n",10*DoublE(a+1),a);
   return 0;}
```

Directiva #define

```
#include <stdio.h>
#define DOUBLE(x) x+x
#define Double(x) (x)+(x)
#define DoublE(x) (x+x)
int main()
{
    int a=3;
    printf("%d, %d\n",10*DOUBLE(a+1),a); //10*a+1+a+1
    printf("%d, %d\n",10*Double(a+1),a); //10*(a+1)+(a+1)
    printf("%d, %d\n",10*DoublE(a+1),a); //10*(a+1+a+1)
    return 0;}
```



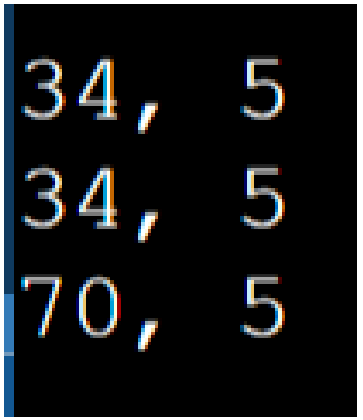
Directiva #define

Ce afișează programul?

```
#include <stdio.h>
#define DOUBLE(x) x+x
#define Double(x) (x)+(x)
#define DoublE(x) (x+x)
int main()
{
   int a=3;
   printf("%d, %d\n",10*DOUBLE(a++),a);
   a=3; printf("%d, %d\n",10*Double(a++),a);
   a=3; printf("%d, %d\n",10*DoublE(a++),a);
   return 0;}
```

Directiva #define

```
#include <stdio.h>
#define DOUBLE(x) x+x
#define Double(x) (x)+(x)
#define DoublE(x) (x+x)
int main()
{ int a=3;
printf("%d, %d\n",10*DOUBLE(a++),a);//10*a++ +(a++)
a=3;
printf("%d, %d\n",10*Double(a++),a); //10*(a++)+(a++)
a=3;
printf("%d, %d\n",10*DoublE(a++),a); //10*((a++)+(a++))
return 0;}
```



Directiva #define

```
#define DOUBLE(x)    ( x ) + ( x )

a = 5;
 printf("%d \n", 10 * DOUBLE( a ) );


          //Echivalent cu:
 a = 5;
 printf("%d \n", 10 * ( a ) + ( a ) ); // => 55
```

Corectie:

```
#define DOUBLE(x)    ( ( x ) + ( x ) )
```

OBS. *Toate macro-urile care evaluează expresii numerice necesită parantezare în aceasta manieră pentru a evita interacțiuni nedorite cu alți operatori*

Directiva #define

- invocarea unui **macro** presupune înlocuirea apelului cu textul macro-ului respectiv
  - ✓ se generează astfel instrucțiuni la fiecare invocare și care sunt ulterior compilate
  - ✓ se recomandă utilizarea doar pentru calcule simple
  - ✓ parametrul formal este înlocuit cu textul corespunzător parametrului actual, corespondența fiind pur pozițională
- timpul de procesare este mai scurt când se utilizează macro-uri (apelul funcției necesită timp suplimentar)

Directiva #define. Exemplu

```c
#include <stdio.h>

//constante simbolice
#define ALPHA 30
#define BETA ALPHA+10
#define GAMMA (ALPHA+10)

//macro-uri
#define MIN(a,b) ((a<b)? a:b)
#define ABS1(x) (x<0)?-x:x
#define ABS2(x) (((x)<0)?(-x):(x))
#define INTERSCHIMB(tip,a,b)\
    {tip c; c=a;a=b;b=c;}
```

```c

int main()
{
    int x=2*BETA, y=2*GAMMA;
    printf("%d, %d\n",x,y);
    int m=MIN(x,y);
    printf("%d, %d\n",m,BETA-x);
    int A=ABS1(BETA-x);
    int A1=ABS1((BETA-x));
    printf("%d, %d\n",A,A1);
    int B=ABS2(BETA-x);
    int B2=ABS2((BETA-x));
    printf("%d, %d\n",B,B2);
    int C=ABS2((x-ALPHA));
    printf("%d, %d, %d\n",A,B,C);
    INTERSCHIMB(int,A,C);
    printf("%d, %d\n",A,C);

    return 0;
}
```

Definiții cu paranteze vs fără paranteze

Directiva #define. Exemplu

```c
#include <stdio.h>

//constante simbolice
#define ALPHA 30
#define BETA ALPHA+10
#define GAMMA (ALPHA+10)

//macro-uri
#define MIN(a,b) ((a<b)? a:b)
#define ABS1(x) (x<0)?-x:x
#define ABS2(x) (((x)<0)?(-x):(x))
#define INTERSCHIMB(tip,a,b)\
    {tip c; c=a;a=b;b=c;}
```

```c

int main()
{
    int x=2*BETA, y=2*GAMMA;
    printf("%d, %d\n",x,y);
    int m=MIN(x,y);
    printf("%d, %d\n",m,BETA-x);
    int A=ABS1(BETA-x);
    int A1=ABS1((BETA-x));
    printf("%d, %d\n",A,A1);
    int B=ABS2(BETA-x);
    int B2=ABS2((BETA-x));
    printf("%d, %d\n",B,B2);
    int C=ABS2((x-ALPHA));
    printf("%d, %d, %d\n",A,B,C);
    INTERSCHIMB(int,A,C);
    printf("%d, %d\n",A,C);

    return 0;
}
```

```
70, 80
70, -30
-90, 30
-90, 30
-90, -90, 40
40, -90
```

ABS1(BETA-x) -> (ALPHA + 10 – x < 0) => -ALPHA + 10 - X (= -30 + 10 - 70 = 10 - 100 = -90)

Directiva #define. Exemplu

```c
#include <stdio.h>
// definire tip date
#define TIP int

// definire cod pe mai multe linii
#define DEBUG_PRINT printf( \
        "File %s, line %d:\n"\
        "x = %d, y = %d, z = %d ", \
        __FILE__ , __LINE__ ,\
        x,y,z)

int main()
{
    TIP x=3,y=5,z;
    x *=2;
    y += x;
    z = x * y;
    DEBUG_PRINT;
return 0;
}
```

//2 macro-uri predefinite

```
File main.c, line 18:
x = 6, y = 11, z = 66
```

Macrodefiniții. *Exemplu*

```c
#include <stdio.h>

#define medie(a, b, c){\
float m = 0; \
m = 0.4 * (a + b) + 0.2 * c \
}

int main()
{
   int  x = 2, y = 3, z = 4;
   medie(x, y, z);
   printf("medie = %.2f ", m);

return 0;
}
```

Macrodefiniții. *Exemplu*

```c
#include <stdio.h>

#define medie(a, b, c){\
float m = 0; \
m = 0.4 * (a + b) + 0.2 * c \
}

int main()
{
   int  x = 2, y = 3, z = 4;
   medie(x, y, z);
   printf("medie = %.2f ", m);

return 0;
}
```

input

Compilation failed due to following error(s).

```
main.c: In function ‘main’:
main.c:6:1: error: expected ‘;’ before ‘}’ token
    6 | }
      | ^
main.c:11:4: note: in expansion of macro ‘medie’
   11 |    medie(x, y, z);
      |    ^~~~~
main.c:12:28: error: ‘m’ undeclared (first use in this function)
   12 |    printf("medie = %.2f ", m);
      |                            ^
```

```c
#include <stdio.h>

#define medie(a, b, c) \
float m = 0; \
m = 0.4 * (a + b) + 0.2 * c \

int main()
{
   int  x = 2, y = 3, z = 4;
   medie(x, y, z);
   printf("medie = %.2f ", m);

return 0;
}
```

```
medie = 2.80

...Program finished with exit code 0
Press ENTER to exit console.
```

Obs. Bloc + declarație de variabilă

Macrodefiniții. *Exemplu*

```c
#include <stdio.h>

#define medie(a, b, c) \
float m = 0; \
m = 0.4 * (a + b) + 0.2 * c \

int main()
{
   int  x = 2, y = 3, z = 4;
   medie(x, y, z);
   printf("medie = %.2f ", m);

return 0;
}
```

```
medie = 2.80
```

```c
#include <stdio.h>

#define medie(a,b,c) 0.4 * (a + b) + 0.2 * c

int main()
{
    int  x = 2, y = 3, z = 4;
    printf("\tmedie = %.2f ", medie(x,y,z));

    return 0;
}
```

input

```
    medie = 2.80
```

Macro versus funcții

Avantaj macro-uri:

Există operații pe care funcțiile nu le pot îndeplini.

De exemplu:

```
#define MALLOC( n, type )    \
    ( (type *) malloc ( ( n ) * sizeof ( type ) ) )

//apel:
    pi = MALLOC( 25, int );
```

Macro versus funcții

Dezavantaj macro-uri: efecte secundare

```
#define MAX( a , b )    ( ( a ) > ( b ) ) ? ( a ) : ( b )
   ...
   x = 5; y = 8; z = MAX( x++ , y++ ) ;
   printf("x = %d, y = %d, z = %d ",x , y , z );

   //sau
   z=((x++) > (y++))?(x++):(y++);
   printf("x = %d, y = %d, z = %d ",x , y , z );
```

Afișare: x = 6, y = 10, z = 9

Macro versus funcții

Proprietate	Macro	Funcție
Dimensiune cod	Codul macro-ului este introdus în program la fiecare apel (program în creștere)	Codul funcției apare o singură dată
Viteza de execuție	Foarte rapid	Timp adițional dat de apel/return
Evaluare argumente	Argumente evaluate cu fiecare folosire în cadrul macro-ului; pot apărea efecte secundare	Argumente evaluate o singură dată (înainte de apel); nu apar efecte secundare datorate evaluărilor multiple
Tip argumente	Macro-urile nu au tip; funcționează cu orice tip de argument compatibil cu operațiile efectuate	Argumentele au tipuri: sunt necesare funcții diferite pentru tipuri diferite de argumente, chiar dacă funcțiile execută același task

Compilarea condiționată

- facilitează dezvoltarea, dar în special testarea codului
- directivele care pot fi utilizate: **#if, #ifdef, #ifndef**
- directiva **`#if`**:

```
#if expr
    text
#endif
```

```
#if expr
    text1
#else  (#elif)
    text2
#endif
```

- ✓ unde **`expr`** este o expresie constantă care poate fi evaluată de către preprocesor,
- ✓ **`text`**, **`text1`**, **`text2`** sunt porțiuni de cod sursă
- ✓ dacă **`expr`** nu este zero atunci **`text`** respectiv **`text1`** sunt compilate, altfel numai **`text2`** este compilat și procesarea continuă dupa **`#endif`**

Compilarea condiționată

```c
#include <stdio.h>

#define VERSION 2

int main()
{
    #if VERSION == 1
    {
        printf("versiunea 1\n");
        printf("Adaugam module pentru versiunea 1\n ");
        //continua cu diverse module
    }
    #elif VERSION == 2
    {
        printf("versiunea 2\n");
        printf("Adaugam module pentru versiunea 2\n ");
        //continua cu diverse alte module
    }
    #elif VERSION == 3
    {
        printf("versiunea 3\n");
        printf("Adaugam module pentru versiunea 3\n ");
        //continua cu alte module
    }
    #endif

    return 0;
}
```

```
versiunea 2
Adaugam module pentru versiunea 2
```

Compilarea condiționată

- directiva **`#ifdef`**:

```
#ifdef nume
    text
#endif
```

```
#ifdef nume
    text1
#else
    text2
#endif
```

- ✓ unde **`nume`** este o constantă care este testată de către preprocesor dacă este definită,
- ✓ **`text, text1, text2`** sunt porțiuni de cod sursă

- ✓ dacă **`nume`** este definită atunci **`text`** respectiv **`text1`** sunt compilate, altfel numai **`text2`** este compilat și procesarea continuă dupa **`#endif`**

Compilarea condiționată

Exemplu

```c
#include <stdio.h>

#define medie(a, b, c) 0.4 * (a + b) + 0.2 * c

int main()
{
    int  x = 2, y = 3, z = 4;

    #ifdef medie
        printf("medie = %.2f ", medie(x, y, z));
    #else
        printf("medie extra= %.2f ", 0.4 * (x + y) + 0.2 * z);
    #endif

    return 0;
}
```

input

```
medie = 2.80
```

Compilarea condiționată

- directiva **`#ifndef`**:

```
#ifndef nume
    text
#endif
```

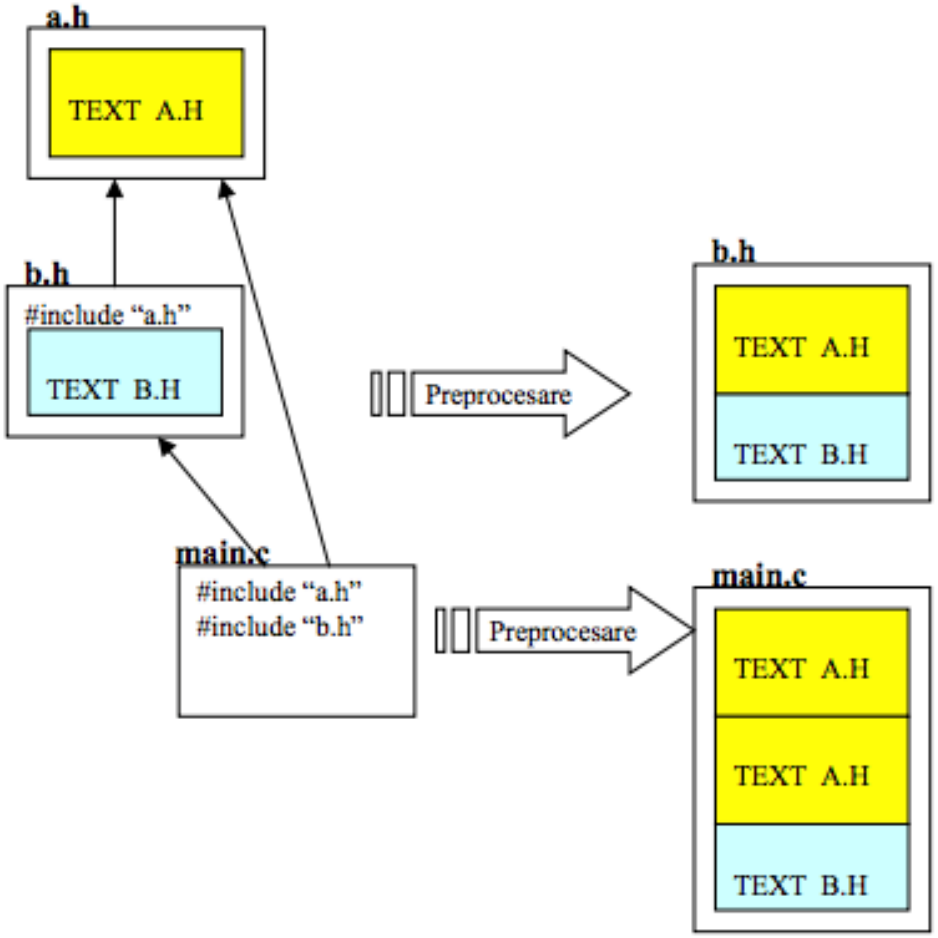
```
#ifndef nume
    text1
#else
    text2
#endif
```

- ✓ unde **`nume`** este o constantă care este testată de către preprocesor dacă NU este definită,
- ✓ **`text`**,**`text1`**,**`text2`** sunt porțiuni de cod sursă

- ✓ dacă **`nume`** NU este definită atunci **`text`** respectiv **`text1`** sunt compilate, altfel numai **`text2`** este compilat și procesarea continuă dupa **`#endif`**

Compilarea condiționată

- directivele **`#ifdef`** și **`#ifndef`** sunt folosite de obicei pentru a evita incluziunea multiplă a modulelor în programarea modulară:
  - fișier antet “a.h”
  - fișier antet “b.h”
    - ✓ include pe “a.h”



- main include “a.h” și “b.h”

Compilarea condiționată

- directivele **`#ifdef`** și **`#ifndef`** sunt folosite de obicei pentru a evita incluziunea multiplă a modulelor în programarea modulară
  - fișier antet “a.h”
  - fișier antet “b.h”

- la începutul fiecărui fișier *header* se practică de obicei o astfel de secvență:

```
#ifndef _MODUL_H_
#define _MODUL_H_
...
#endif /* _MODUL_H_ */
```

Macro-uri predefinite

Există o serie de macro-uri predefinite care nu trebuie re/definite:

`__DATE__`	- data compilării
`__CDECL__`	- apelul funcției urmărește convențiile C
`__STDC__`	- definit dacă trebuie respectate strict regulile ANSI – C
`__FILE__`	- numele complet al fișierului curent compilat
`__FUNCTION__`	- numele funcției curente
`__LINE__`	- numărul liniei curente

Se scriu câte două simboluri underscore!

Macro-uri predefinite. *Exemplu*

```c
#include <stdio.h>
//constante simbolice
#define DEBUG
#define X -3
#define Y 5

int main()
{
    #ifdef DEBUG
        printf("Suntem in functia %s.\n", __FUNCTION__);
    #endif

    #if X + Y
        double a = 3.1;
    #else
        double a = 5.7;
    #endif

    a *= 2;

    #ifdef DEBUG
        printf("La linia %d valoarea lui a este %f\n", __LINE__, a);
    #endif

    a +=10;
    printf("a este %f", a);

    return 0;
}
```

```
Suntem in functia main.
La linia 22 valoarea lui a este 6.200000
a este 16.200000
```

Cuprinsul cursului de azi

1. Directive de preprocesare. Macrodefiniții
2. **Funcții de citire/ scriere**

Funcții de citire/ scriere

Cuprins:

- Funcțiile getchar și putchar
- Funcțiile gets/ fgets și puts
- Funcția printf. Modelatori de format
- Funcția scanf

Funcții de citire și scriere

- operații de **citire** și **scriere** în C:
  - ✓ de la **tastatură** (stdin) și la **ecran** (stdout)
  - ✓ **prin fișiere**
  - ✓ efectuate cu ajutorul funcțiilor de bibliotecă

- **citirea de la tastatură** și **scrierea la ecran**
  - ✓ fără formatare:

    getchar, putchar, getch, getche, putch, gets, puts
  - ✓ cu formatare: scanf, printf
  - ✓ incluse în biblioteci:

    stdio.h : getchar, putchar, gets, puts, scanf, printf

    conio.h : getch, getche, putch

Obs. CODE::BLOCKS nu include biblioteca conio.h

Funcțiile getchar și putchar

- operații de **citire** și **scriere** a caracterelor:
  - ✓ **int getchar(void)**
    - ➢ citește un caracter de la tastatură
    - ➢ așteaptă până este apăsată o tastă și returnează valoarea sa → tasta apăsată are imediat ecou pe ecran
  - ✓ **int putchar(int c)**
    - ➢ scrie un caracter pe ecran în poziția curentă a cursorului
  - ✓ fișierul antet pentru aceste funcții: **stdio.h**.

Funcțiile gets și puts

- operații de **citire** și **scriere** a șirurilor de caractere:
  - ✓ **char *gets(char *s)**
    - ➢ citește caractere din **stdin** și le depune în zona de date de la adresa s, până la apăsarea tastei Enter. În șir, tastei Enter îi va corespunde caracterul '\0'.
    - ➢ **dacă operația de citire reușește, funcția întoarce adresa șirului, altfel valoarea NULL (= 0)**
  - ✓ **int puts(const char *s)**
    - ➢ scrie pe ecran șirul de la adresa s sau o constantă șir de caractere și apoi trece la linie nouă.
    - ➢ **dacă operația de scriere reușește, funcția întoarce ultimul caracter, altfel valoarea EOF (-1).**
  - ✓ fișierul antet pentru aceste funcții este **stdio.h**

Funcțiile gets și puts

```c
#include <stdio.h>

int main()
{
    char sir[10];

    printf("Ce zi este astazi? ");
    gets(sir);
    puts(sir);

    puts("Alegeti Da sau NU. Optiunea dvs este: ");
    gets(sir);
    puts(sir);

    return 0;
}
```

```
  667 | extern char *gets (char *__s) __wur __attribute_deprecated__;
      |             ^~~~
/nix/store/7h3qnwgvkw6z2r81q4j5mks416r5x2cq-binutils-2.44/bin/ld: /tmp/
cc2M8usJ.o: in function `main':
main.c:(.text+0x24): warning: the `gets' function is dangerous and shou
ld not be used.
Ce zi este astazi? Luni
Luni
Alegeti Da sau NU. Optiunea dvs este:
Da
Da
```

Funcția gets. Dezavantaje

- **char *gets(char *s)**
  - ✓ primește ca input numai un buffer (s), nu știm dimensiunea lui
  - ✓ problema de buffer overflow: citim în s mai mult decât dimensiunea lui, gets nu ne împiedică, scrie datele în altă parte
- **Soluție:** funcția fgets:

  char *fgets(char *s, int size, FILE *stream)

  fgets(buffer, sizeof(buffer), stdin);

Obs. În standardul C11 functia gets este eliminată.

Funcțiile fgets, printf și puts

```c
#include <stdio.h>

int main()
{
    char sir[10];

    printf("Ce zi este astazi? ");
    fgets(sir, sizeof(sir), stdin);
    puts(sir);

    puts("Alegeti Da sau NU. Optiunea dvs este: ");
    fgets(sir, sizeof(sir), stdin);
    puts(sir);

    return 0;
}
```

trece la linie nouă

input

```
Ce zi este astazi? LUNI
LUNI

Alegeti Da sau NU. Optiunea dvs este:
DA
DA
```

Funcțiile printf și scanf

- funcții de citire (scanf) și scriere (printf) cu formatare;
- formatarea specifică conversia datelor de la reprezentarea externă în reprezentarea internă (scanf) și invers (printf);
- formatarea se realizează pe baza descriptorilor de format

%[flags][width][.precision][length]specifier

- detalii aici: http://www.cplusplus.com/reference/cstdio/printf/

Funcțiile printf și scanf

- formatarea se realizează pe baza descriptorilor de format

%[flags][width][.precision][length]specifier

Specificator de format	Reprezentare
%c	caracter
%s	șir de caractere
%d, %i	întreg în zecimal
%u	întreg în zecimal fără semn
%o	întreg în octal
%x	întreg în hexazecimal fără semn (litere mici)
%X	întreg în hexazecimal fără semn (litere mari)
%f	număr real în virgulă mobilă
%e, %E	notație științifică - o cifră la parte întreagă
%ld, %li, %lu, %lo, %lx	cu semnificațiile de mai sus, pentru întregi lungi
%p	pointer

Funcția printf

- prototipul funcției:

  int printf(const char *format, argument1, argument2, …);

unde:

- ✓ ***format*** este un șir de caractere ce definește textele și formatele datelor care se scriu pe ecran
- ✓ ***argument1***, ***argument2***,... sunt expresii. Valorile lor se scriu pe ecran conform specificatorilor de format prezenți în format

- functia **printf** întoarce numărul de octeți transferați sau EOF (-1) în caz de eșec.

Funcția printf

```c
#include <stdio.h>

int main()
{
    char sir[] = "Azi e luni.";
    printf("Primul caracter din sirul \"%s\" este %c.\n", sir, sir[0]);

    int x = 1234;
    printf("\nReprezentarea lui x:");
    printf("\nin baza 10: x=%d", x);
    printf("\nin baza 8: x=%o", x);
    printf("\nin baza 16 (litere mici): x=%x", x);
    printf("\nin baza 16 (litere mari): x=%X", x);

    float y = 12.34;
    printf("\n\nReprezentarea lui y:");
    printf("\nca numar real: y=%f", y);
    printf("\nin notatie stiintifica: y=%e", y);
    printf("\nin notatie stiintifica: y=%E", y);

    return 0;
}
```

Funcția printf

```c
#include <stdio.h>

int main()
{
    char sir[] = "Azi e luni.";
    printf("Primul caracter din sirul \"%s\" este %c.\n", sir, sir[0]);

    int x = 1234;
    printf("\nReprezentarea lui x:");
    printf("\nin baza 10: x=%d", x);
    printf("\nin baza 8: x=%o", x);
    printf("\nin baza 16 (litere mici): x=%x", x);
    printf("\nin baza 16 (litere mari): x=%X", x);

    float y = 12.34;
    printf("\n\nReprezentarea lui y:");
    printf("\nca numar real: y=%f", y);
    printf("\nin notatie stiintifica:
    printf("\nin notatie stiintifica:

    return 0;
}
```

```
Primul caracter din sirul "Azi e luni." este A.

Reprezentarea lui x:
in baza 10: x=1234
in baza 8: x=2322
in baza 16 (litere mici): x=4d2
in baza 16 (litere mari): x=4D2

Reprezentarea lui y:
ca numar real: y=12.340000
in notatie stiintifica: y=1.234000e+01
in notatie stiintifica: y=1.234000E+01
```

Funcția printf

```c
#include <stdio.h>

int main()
{
    int a = printf("Ce zi este astazi? ");
    printf("\na = %d \n", a);

    a = printf("Alegeti Da sau NU. Optiunea dvs este: ");
    printf("\na = %d \n", a);

    return 0;
}
```

input

```
Ce zi este astazi?
a = 19
Alegeti Da sau NU. Optiunea dvs este:
a = 38
```

Număr de octeți:

Functia **printf** întoarce numărul de octeți transferați sau EOF (-1) în caz de eșec.

Modelatori de format

- mulți specificatori de format pot accepta modelatori care modifică ușor semnificația lor

- modelatorul de format se află între semnul procent și codul pentru format:
  - ✓ alinierea la stânga: caracterul '–'
  - ✓ minim de mărime a câmpului: șir de cifre zecimale
  - ✓ numărul de cifre zecimale (precizia reprezentării): caracterul '.' urmat de cifre

Modelatori de format. *Exemplu*

```c
#include <stdio.h>

int main()
{
    int a = 1234;
    printf("\na = %8d aliniat la dreapta\n", a);
    printf("\na = %-8d aliniat la stanga\n", a);

    float x = 12.34;
    printf("\nx = %7.2f \n", x);

    return 0;
}
```

```
a =     1234 aliniat la dreapta

a = 1234     aliniat la stanga

x =   12.34
```

Modelatorul flags

- formatarea se realizează pe baza descriptorilor de format

%[flags][width][.precision][length]specifier

flags	description
-	Left-justify within the given field width; Right justification is the default (see *width* sub-specifier).
+	Forces to preceed the result with a plus or minus sign (+ or -) even for positive numbers. By default, only negative numbers are preceded with a - sign.
(space)	If no sign is going to be written, a blank space is inserted before the value.
#	Used with o, x or X specifiers the value is preceeded with 0, 0x or 0X respectively for values different than zero. Used with a, A, e, E, f, F, g or G it forces the written output to contain a decimal point even if no more digits follow. By default, if no digits follow, no decimal point is written.
0	Left-pads the number with zeroes (0) instead of spaces when padding is specified (see *width* sub-specifier).

Modelatorul width

- formatarea se realizează pe baza descriptorilor de format

%[flags][width][.precision][length]specifier

width	description
(number)	Minimum number of characters to be printed. If the value to be printed is shorter than this number, the result is padded with blank spaces. The value is not truncated even if the result is larger.
*	The *width* is not specified in the *format* string, but as an additional integer value argument preceding the argument that has to be formatted.

Modelatorul precision

- formatarea se realizează pe baza descriptorilor de format

%[flags][width][.precision][length]specifier

.precision	**description**
.number	For integer specifiers (`d`, `i`, `o`, `u`, `x`, `X`): *precision* specifies the minimum number of digits to be written. If the value to be written is shorter than this number, the result is padded with leading zeros. The value is not truncated even if the result is longer. A *precision* of `0` means that no character is written for the value `0`. For `a`, `A`, `e`, `E`, `f` and `F` specifiers: this is the number of digits to be printed **after** the decimal point (by default, this is 6). For `g` and `G` specifiers: This is the maximum number of significant digits to be printed. For `s`: this is the maximum number of characters to be printed. By default all characters are printed until the ending null character is encountered. If the period is specified without an explicit value for *precision*, `0` is assumed.
.*	The *precision* is not specified in the *format* string, but as an additional integer value argument preceding the argument that has to be formatted.

Modelatorul length

- formatarea se realizează pe baza descriptorilor de format

%[flags][width][.precision][length]specifier

	specifiers						
length	**d i**	**u o x X**	**f F e E g G a A**	**c**	**s**	**p**	**n**
(none)	int	unsigned int	double	int	char*	void*	int*
hh	signed char	unsigned char					signed char*
h	short int	unsigned short int					short int*
l	long int	unsigned long int		wint_t	wchar_t*		long int*
ll	long long int	unsigned long long int					long long int*
j	intmax_t	uintmax_t					intmax_t*
z	size_t	size_t					size_t*
t	ptrdiff_t	ptrdiff_t					ptrdiff_t*
L			long double				

Modelatori de format pentru printf

main.c

```c
#include <stdio.h>
#include <stdlib.h>


int main(){

    double numar;
    numar = 10.1234;

    printf("numar=%f\n",numar);
    printf("numar=%10f\n",numar);
    printf("numar=%012f\n",numar);

    printf("%.4f\n",123.1234567);
    printf("%3.8d\n",1000);
    printf("%10d\n",1000);
    printf("%-10d\n",1000);
    printf("%10.15s\n","Acesta este un test simplu");

    return 0;
}
```

```
numar=10.123400
numar= 10.123400
numar=00010.123400
123.1235
00001000
      1000
1000
Acesta este un
```

Funcția printf

Exemplu:

```
printf("valoarea lui x este: %-4.2f\n",3.14);
printf("x=%i, y=%f, x=%o, x=%#x\n",15,3.14,15,15);
printf("c= %c, c=%d\n",'%','%');
printf("sir de caractere: %s\n", "ana are mere");
printf("\\ \" \' \n");
```

```
valoarea lui x este: 3.14
x=15, y=3.140000, x=17, x=0xf
c= %, c=37
sir de caractere: ana are mere
\ " '
```

Funcția scanf

- prototipul funcției:

int scanf(const char * format ,adresa1, adresa2, …);

unde:

- ✓ ***format*** este un șir de caractere ce definește textele și formatele datelor care se citesc de la tastatură
- ✓ ***adresa1***, ***adresa2***,... sunt adresele zonelor din memorie în care se păstrează datele citite după ce au fost convertite din reprezentarea lor externă în reprezentare internă.

- funcția **scanf** întoarce numărul de câmpuri citite si depuse la adresele din listă. Dacă nu s-a stocat nici o valoare, funcția întoarce 0.

Funcția scanf

- șirul de formatare (format) poate include următoarele elemente:
  - ✓ **spațiu alb**: funcția citește și ignoră spațiile albe (spațiu, tab, linie nouă) înaintea următorului caracter diferit de spațiu
  - ✓ **caracter diferit de spațiu, cu excepția caracterului %:** funcția citește următorul caracter de la intrare și îl compară cu caracterul specificat în șirul de formatare
    - ➢ dacă se potrivește, funcția are succes și trece mai departe la citirea următorului caracter din intrare
    - ➢ dacă nu se potrivește, funcția eșuează și lasă următoarele caractere din intrare nepreluate

Cursul 5

1. Directive de preprocesare. Macrodefiniții
2. Funcții de citire / scriere

Cursul 6

1. **Fișiere text. Funcții specifice de manipulare**
2. **Funcții**